

Final Project Report & Project Documentation

Project Title: Student Course Registration System

Methodology: Agile Kanban (Trello)	Project Duration: 3 Weeks (14 Days)	Team Size: 6 Members (Students A-F)
--	---	---

1. Executive Summary

This document serves as the comprehensive final project report for the development of the **Student Course Registration System**. Over a three-week lifecycle, our team successfully planned, designed, developed, and tested a functional software solution utilizing Agile Kanban principles. By structuring roles clearly, applying strict Work-in-Progress (WIP) limits, and tracking workflow metrics via Trello, the team managed to optimize throughput, resolve critical blockers collaboratively, and deliver all core features on time.

2. Team Structure & RACI Responsibility Matrix

To maximize accountability and prevent overlap, the team was structured around specialized cross-functional roles. Responsibility allocations for each core deliverable were guided by a strict RACI matrix:

Team Roles

- Student A (Kanban Manager):** Accountable for daily board health, monitoring WIP limits, tracking metrics, and final documentation.
- Student B (Business Analyst):** Responsible for tracking requirements, compiling use cases, and supporting system testing.
- Student C (Database Designer):** Responsible for entity relationship diagrams (ERD) and relational SQL database schemas.
- Student D (UI/UX Designer):** Responsible for creating user interface wireframes and interactive prototypes.
- Student E (Lead Developer):** Responsible for core code architecture and feature module engineering.
- Student F (QA Tester):** Responsible for collaborative programming, writing test cases, logging bugs, and validating features.

RACI Matrix

Deliverable / Task	A	B	C	D	E	F
Requirements Gathering	A	R	C	C	C	C

Deliverable / Task	A	B	C	D	E	F
Use Case Diagram	I	R	C	C	I	I
Database Design & ERD	I	C	R	I	C	I
UI Wireframes	A	C	I	R	C	I
Login & Auth Module	A	I	C	C	R	C
Course Registration Module	A	I	C	I	R	R
Drop Course Module	A	I	C	I	C	R
View Registered Courses	A	I	C	C	C	R
System Testing & Bug Logging	A	R	C	I	C	R
Project Documentation	R	C	C	C	C	C

R = Responsible | A = Accountable | C = Consulted | I = Informed

3. Kanban Workflow Architecture & Board Setup

Column Strategy & WIP Controls

To establish an optimal flow and avoid operational congestion, the group operated across a 5-stage pipeline with embedded Work-In-Progress (WIP) boundaries:

1. **Backlog:** All 10 high and medium-priority project tasks initialized here (Unlimited WIP).
2. **To Do:** Scope committed for the current weekly iteration (Unlimited WIP).
3. **In Progress:** Maximum of **3 tasks** allowed simultaneously. This strict limit forced the team to finish existing work before taking on more, encouraging peer programming when roadblocks appeared.
4. **Testing:** Maximum of **2 tasks** under evaluation by the QA Tester at one time. This kept a steady flow of verified features moving toward completion.
5. **Done:** Completed, reviewed, and fully accepted components (Unlimited WIP).

Visual Signaling System (Labels)

- **Red:** High Priority Features (Login, Course View, Course Registration)
- **Yellow:** Medium Priority Features (Drop Courses, View Registered Courses)
- **Green:** Low Priority
- **Blue:** Project Documentation
- **Purple:** UI/UX & Wireframe Design
- **Orange:** System Bugs & Issues

4. Group Performance & Flow Metrics

Cumulative Flow Trends

The distribution of tasks across the 14-day development window illustrates a predictable, healthy workflow progression:

Timeline	Backlog	To Do	In Progress	Testing	Done
Day 1	10	0	0	0	0
Day 3	7	2	1	0	0
Day 5	5	2	3	0	0
Day 7	3	1	3	1	2
Day 10	1	0	2	2	5
Day 14	0	0	0	0	10

Cycle Time Breakdown

Cycle time was tracked diligently from the moment work began on a card until it was fully signed off in the “Done” column:

- **Requirements Gathering:** Day 1 to Day 2 (2 days)
- **Use Case Diagram:** Day 2 to Day 3 (1 day)
- **Database Design:** Day 3 to Day 5 (2 days)
- **UI Wireframes:** Day 3 to Day 5 (2 days)
- **Login Module:** Day 6 to Day 8 (3 days)
- **Registration Module:** Day 7 to Day 10 (4 days)
- **Drop Module:** Day 9 to Day 11 (2 days)
- **View Courses:** Day 9 to Day 10 (1 day)
- **Testing:** Day 11 to Day 13 (3 days)
- **Documentation:** Day 12 to Day 14 (2 days)

Group Average Cycle Time: 2.2 Days per Task

5. Risk Management & Working Agreements

Blocker Resolutions

- **Database Availability:** When backend architecture was still being designed, developers used temporary JSON and mock data to avoid halting UI/UX progress.
- **WIP Limit Bottlenecks:** When active tasks hit the column cap of 3, developers paused new work and paired up to finish open modules first.
- **Late Regression Bugs:** When QA uncovered late defects, tasks were kept from moving backwards. Instead, an orange Bug Card was logged in the backlog and immediately prioritized.

Team Working Agreements

1. **Rule 1:** WIP Limits must never be violated ($\text{In Progress} \leq 3$, $\text{Testing} \leq 2$).
2. **Rule 2:** Cards cannot move backwards on the board; new bug cards must be logged instead.
3. **Rule 3:** Every card must have a designated owner, an explicit due date, and at least 3 checklist items.
4. **Rule 4:** Daily standup sessions are capped at a maximum of 15 minutes.
5. **Rule 5:** No card reaches “Done” without its checklist completed and QA signing off.
6. **Rule 6:** Technical or scheduling blockers must be flagged immediately on Trello.

6. Project Retrospective & Strategic Reflection

Adopting the Agile Kanban framework was essential to our group’s success in delivering the Student Course Registration System within the 3-week timeline.

Initially, the team struggled to adapt to the strict WIP limits, particularly between Day 3 and Day 5 when multiple tasks were in flight simultaneously. However, this constraint quickly proved its value. Rather than allowing individual team members to work in silos, the WIP limits encouraged a collaborative environment where developers and testers stepped in to help clear blocked tasks.

The metrics tell a clear story of our improvement: keeping an average cycle time of 2.2 days ensured that design, development, and testing phases remained well-integrated rather than piling up at the very end of the project. Daily standups kept communication clear and fast, allowing the team to quickly resolve issues like database dependencies and late-stage bugs. Ultimately, this project demonstrated how visual workflow management, shared accountability, and clear team agreements can help a cross-functional group deliver high-quality software on schedule.